



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Strategie di ricerca non informata

Ingegneria Informatica

Docente

Domenico Fabio Savo

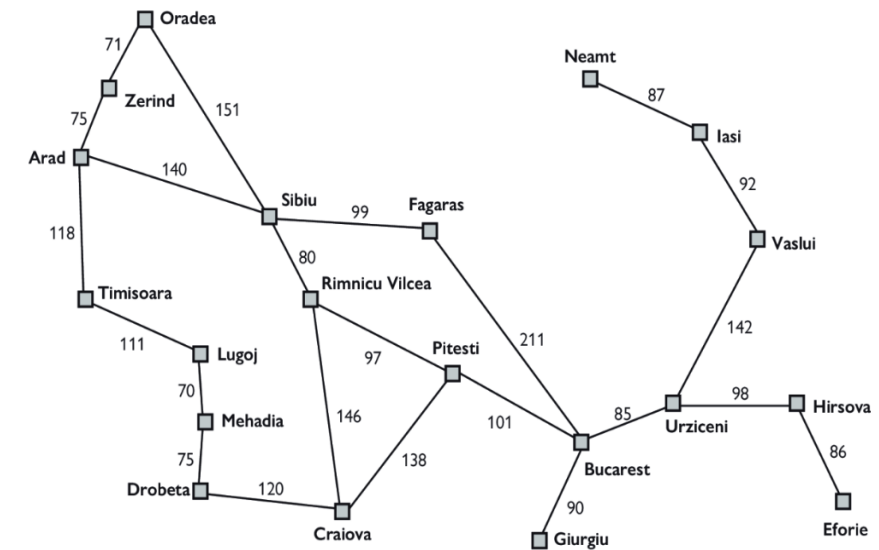
Intelligenza Artificiale

Ricerca non Informata

Un algoritmo di **ricerca non informata** non riceve alcuna informazione su quanto uno stato sia vicino all'obiettivo.

Consideriamo il nostro agente che da Arad deve raggiungere Bucarest:

- Un agente **non informato** non conosce la mappa della Romania e non ha quindi modo di sapere se sia meglio come primo passo andare a Zerind o Sibiu.
- Un **agente informato**, invece, conosce la posizione di ogni città e sa che Sibiu è molto più vicina a Bucarest e può quindi individuare il cammino più breve più facilmente



Ricerca per ampiezza

Quando tutte le azioni hanno lo stesso costo, una strategia appropriata è la **ricerca in ampiezza**: si espande prima il nodo radice, poi tutti i suoi successori, poi i successori di questi ultimi, e così via.

Si tratta di una ricerca sistematica, quindi **completa** anche su spazi degli stati infiniti.



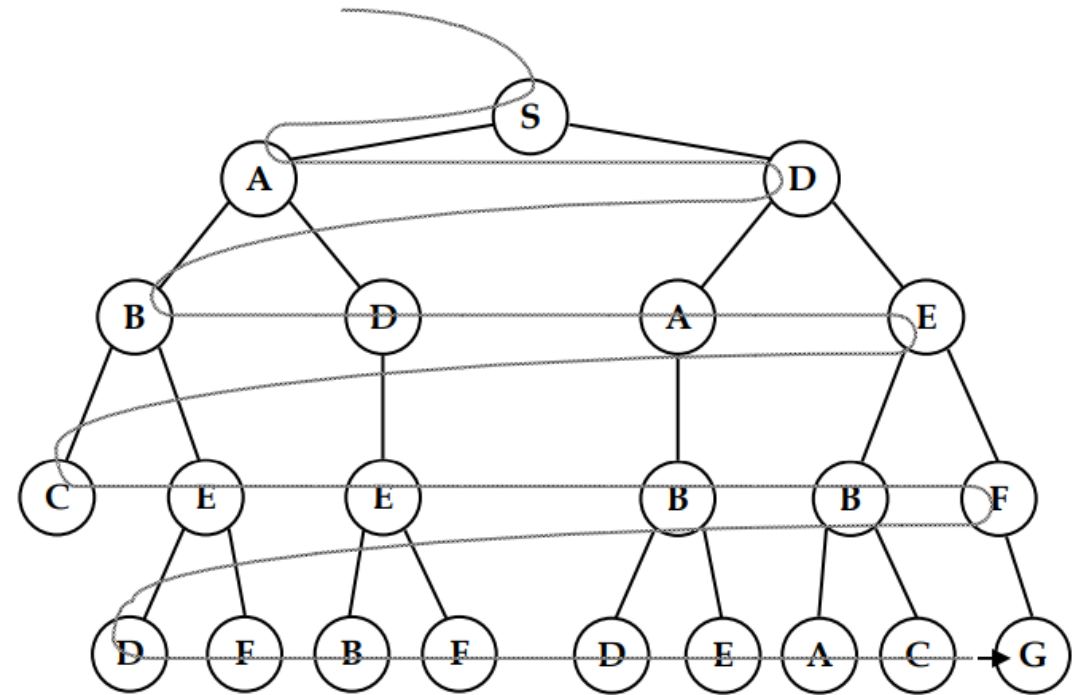
Ricerca in ampiezza: Algoritmo

```
function Ricerca-In-Ampiezza(problema) returns un nodo soluzione o fallimento
  nodo ← NODO(problema.StatoIniziale)
  if nodo.Stato == problema.Obiettivo then return nodo
  frontiera ← una coda FIFO, con nodo come elemento iniziale
  raggiunti ← {problema.StatoIniziale}
  while not Vuota?(frontiera) do
    nodo ← Pop(frontiera)
    for each figlio in Espandi(problema, nodo) do
      s ← figlio.Stato
      if s == problema.Obiettivo then return figlio
      if s not in raggiunti then
        add s a raggiunti
        add figlio a frontiera
  return fallimento
```



Ricerca in ampiezza: esempio

Tutti i nodi di profondità **d** nell'albero si **espandono** prima dei nodi di profondità **d+1**



Ricerca per ampiezza: osservazioni

- L'algoritmo utilizza una **coda FIFO** per implementare la *frontiera*
- Si utilizza nei casi in cui tutti gli archi (azioni) hanno **lo stesso costo** pari ad 1
- Se esiste una sola soluzione, la *ricerca in ampiezza* la trova (*completezza*)
- Se esistono più soluzioni, viene trovato per prima lo stato obiettivo più alto/superficiale e quindi a costo minimo (*ottimalità*)

Valutazione: ricerca **completa** ed **ottimale**



Ricerca per ampiezza: Complessità

In termini di tempo e spazio, supponiamo di effettuare una ricerca in un albero uniforme dove ogni stato ha b successori (**fattore di ramificazione**).

La radice dell'albero di ricerca genera quindi b nodi, ognuno dei quali genera altri b nodi, per un totale di b^2 al secondo livello. Ognuno di questi genera altri b nodi, per un totale di b^3 al terzo livello, e così via. Ora supponiamo che la soluzione sia a profondità d . Allora il numero totale di nodi generati è:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$$

Tutti i nodi rimangono in memoria, perciò le complessità **temporale** e **spaziale** sono entrambe $O(b^d)$ quindi **esponenziali** quindi possono considerarsi decisamente non buone



Ricerca per ampiezza: esempio di costi

Supponiamo di dover effettuare una ricerca su un albero con fattore di ramificazione $b=10$ e supponiamo che il programma sia in grado di elaborare 1000 nodi al secondo, e che richieda 100 byte per nodo.

Livello	Nodi	Tempo	Memoria
2	111	0,1 sec	11 Kb
6	10^6	18 min	111 Mb
10	10^{10}	128 giorni	11 Gb
14	10^{14}	3500 anni	11.111 Tb

NB: 1 Tb - 1000 miliardi di byte



Ricerca a costo uniforme

Quando le azioni hanno **costi diversi**, la strategia ovvia è di usare una **ricerca best-first** in cui la funzione di valutazione **f** è il costo del cammino dalla radice al nodo corrente (**noto.Costo-Cammino**)

I **Costi** possono essere le distanze tra i punti rappresentati dai nodi, oppure i costi possono essere le risorse richieste dall'agente per realizzare il task associato ad un nodo.

Questa topo di ricerca è anche nota come algoritmo di **Dijkstra**.

*L'idea è che mentre la ricerca in ampiezza si diffonde a ondate di profondità uniforme – prima profondità 1, poi 2 e così via – la ricerca a costo uniforme si diffonde a **ondate di costo uniforme**.*



Ricerca a costo uniforme: Algoritmo

L'algoritmo può essere implementato come chiamata di **Ricerca-Best-First** con **Costo-Cammino** come **funzione di valutazione**

```
function Ricerca-Costo-Uniforme(problema) returns un nodo soluzione o fallimento  
    return Ricerca-Best-First(problema, Costo-Cammino)
```

In questo modo, i nodi della *frontiera* saranno ordinati secondo il loro costo. Come conseguenza i nodi con costo inferiore saranno **espansi per primi**.

Il costo del cammino di un nodo è la somma dei costi degli operatori che determinano il cammino.



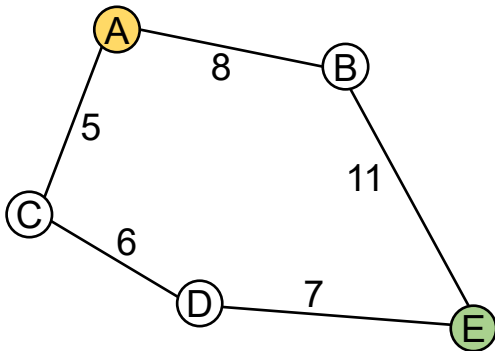
Algoritmo di ricerca Best-first (recap.)

```
function Ricerca-Best-First(problema, f) returns un nodo soluzione o fallimento  
  nodo  $\leftarrow$  Nodo(Stato=problema.StatoIniziale)  
  frontiera  $\leftarrow$  una coda con priorità ordinata in base a f, con nodo come elemento iniziale  
  raggiunti  $\leftarrow$  una tabella di lookup, con un elemento con chiave problema.StatoIniziale e valore nodo  
  while not Vuota?(frontiera) do  
    nodo  $\leftarrow$  Pop(frontiera)  
    if nodo.Stato == problema.Obiettivo then return nodo  
    for each figlio in Espandi(problema, nodo) do  
      s  $\leftarrow$  figlio.Stato  
      if s not in raggiunti or figlio.Costo-Cammino < raggiunti[s].Costo-Cammino then  
        raggiunti[s]  $\leftarrow$  figlio  
        aggiungi figlio a frontiera  
  
  return fallimento
```



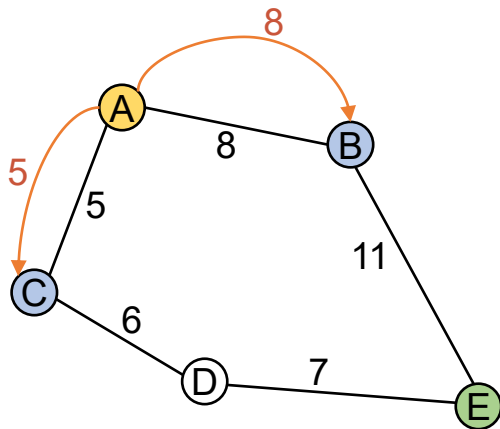
Ricerca a costo uniforme: esempio

Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



Ricerca a costo uniforme: esempio

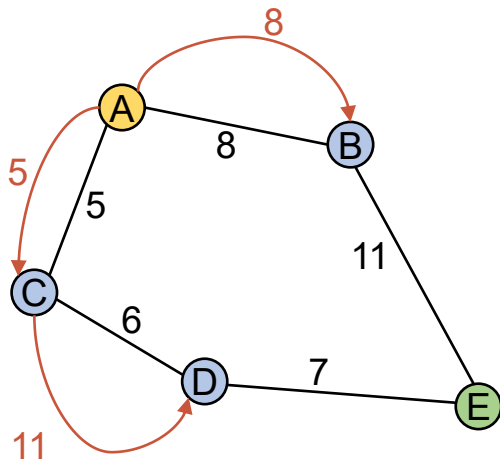
Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



Il primo nodo ad essere espanso è il nodo **A** da cui è possibile raggiungere in nodi **B** (con costo 8) e **C** (con costo 5).

Ricerca a costo uniforme: esempio

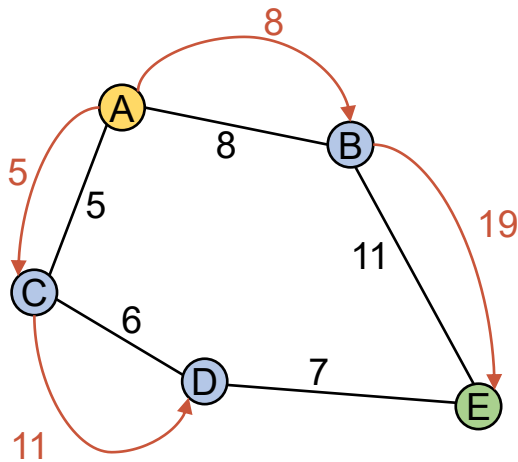
Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



Tra **B** e **C** il percorso con nodo inferiore è quello che va verso **C**. **C** quindi viene scelto per essere espanso. Questo percorso porterà l'agente al nodo **D** (con **costo 5 + 6 = 11**).

Ricerca a costo uniforme: esempio

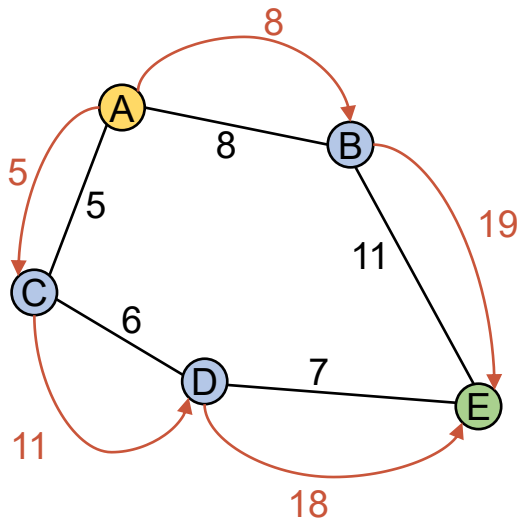
Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



Tra i nodi nella frontiera (**B** e **D**) il percorso verso **B** è il meno costoso. L'algoritmo sceglie quindi di espandere **B** raggiungendo il nodo **E** (con **costo 8 + 11 = 19**). Il nodo **E** viene inserito nella frontiera.

Ricerca a costo uniforme: esempio

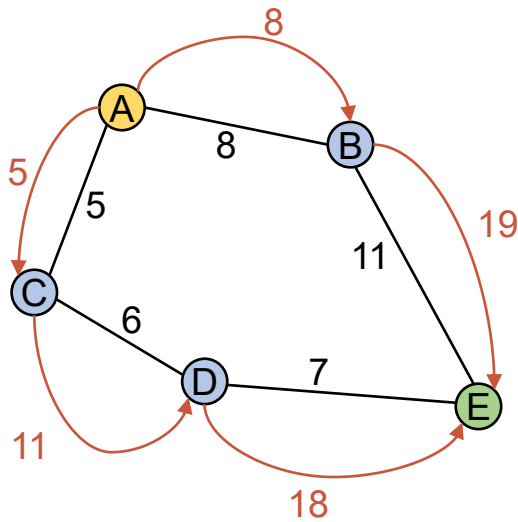
Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



Tra i nodi nella frontiera (**D** e **E**) il percorso verso **D** è il meno costoso. L'algoritmo sceglie quindi di espandere **D** raggiungendo il nodo **E** (con **costo** $11 + 7 = 18$). Il costo del nodo **E**, che si trova già nella frontiera, viene aggiornato passando da **19** ad **18**.

Ricerca a costo uniforme: esempio

Supponiamo che un agente partendo dal **nodo A** (stato iniziale) debba raggiungere il **nodo E** (stato obiettivo) **minimizzando la distanza percorsa**. La distanza tra due nodi adiacenti è indicata sull'arco che li unisce.



La frontiera contiene ora il solo nodo **E** che viene estratto e riconosciuto come **stato obiettivo**. L'algoritmo termina restituendo **E** con un costo di **18**

Ricerca a costo uniforme: osservazioni e complessità

- L'algoritmo utilizza una **coda con priorità**, in cui viene estratto prima il nodo di costo minimo per implementare la **frontiera**
- Si utilizza in caso di costi di passo non uguali nell'albero di ricerca.
- Se esiste una sola soluzione, la **ricerca a costo uniforme** la trova (**completezza**)
- Se esistono più soluzioni, viene trovato per prima lo stato obiettivo a costo minimo (**ottimalità**) assumendo che costo per raggiungere un nodo sia una **funzione non decrescente** in base alla sua profondità.

Valutazione: ricerca **completa** ed **ottimale**

La **complessità** della ricerca a **costo uniforme** dipende dal costo dei cammini e non dalla loro profondità, come nel caso della ricerca in ampiezza. La sua complessità può essere anche **maggiore di b^d** (quindi non buona)



Ricerca in profondità

La **ricerca in profondità** espande sempre il primo dei nodi fino a raggiungere il livello più profondo dell'albero. Nodi di uguale profondità vengono selezionati arbitrariamente (di solito quelli più a sinistra).

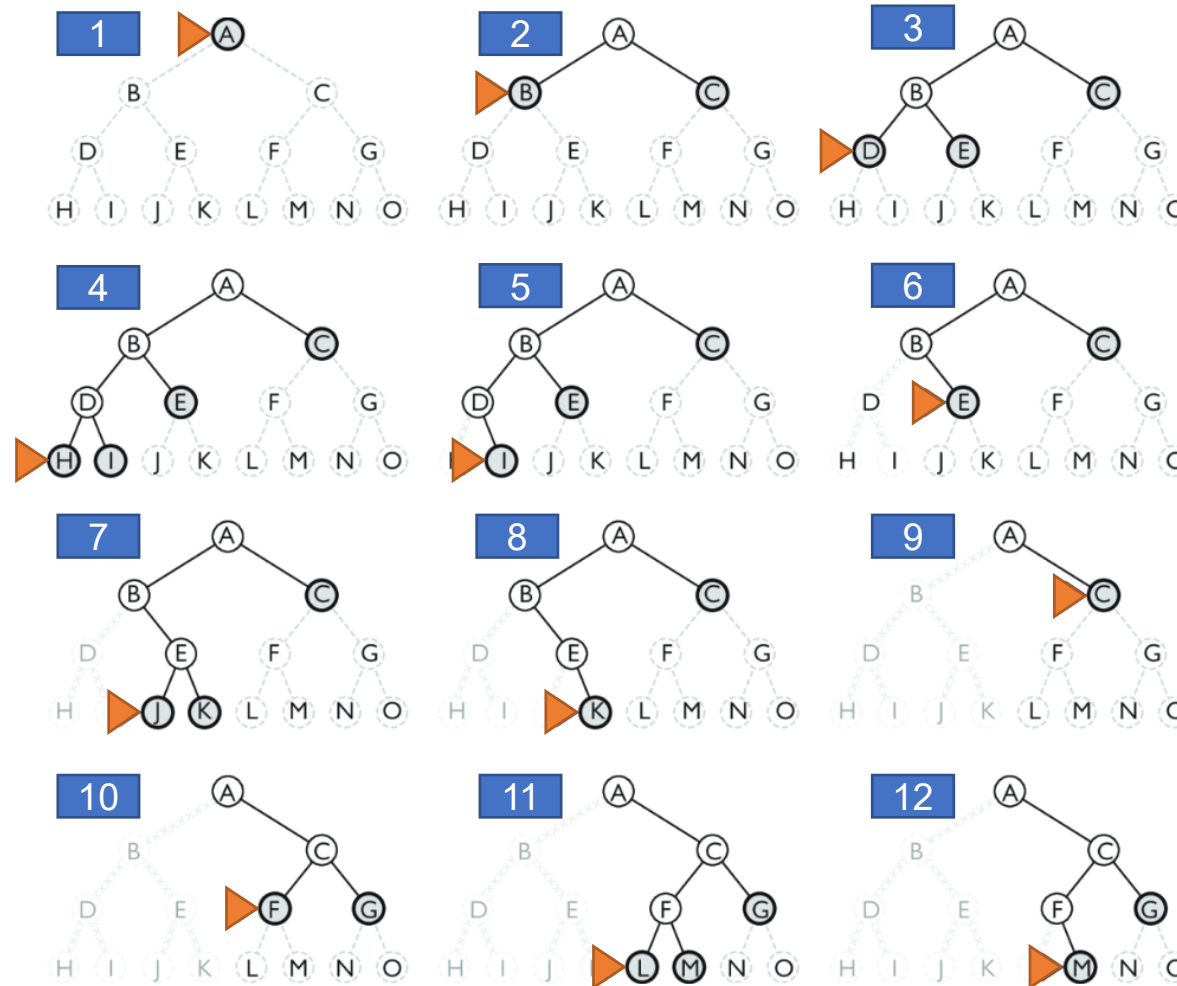
La ricerca torna indietro (backtracking) ed espande nodi a livelli più superficiali solo quando arriva ad un **nodo foglia** non obiettivo.

La funzione di ricerca userà un meccanismo di inserimento in una coda LIFO (o **stack**) dove l'elemento estratto è sempre l'ultimo che era stato inserito.

Generalmente questa ricerca è implementata non come ricerca su grafo ma come ricerca ad albero che **non mantiene** una tabella degli stati raggiunti.



Ricerca in profondità: esempio



Ricerca in profondità: osservazioni

- L'algoritmo utilizza una **coda LIFO** per implementare la *frontiera*
- **Ottimalità**: non è ottimo perché non usa nessun criterio per favorire le soluzioni a costo minore e quindi la prima soluzione trovata può non essere la soluzione ottima.
- **Completezza**: non è completo perché può imbattersi in un percorso di profondità infinita.

Valutazione: ricerca né **completa** né **ottimale**

Complessità:

- Se **b** è il fattore di ramificazione e **m** è la massima profondità dell'albero, allora il numero massimo di nodi espansi è: $1 + b + b^2 + b^3 + \dots + b^m$
- **Complessità temporale**: $O(b^m)$
- **Complessità spaziale**: $O(b \cdot m)$ (inferiore alla ricerca per ampiezza). Alcuni problemi che richiederebbero *exabyte* (**1.024 petabyte**) di memoria con la ricerca in ampiezza possono essere affrontati con pochi *kilobyte* usando la ricerca in profondità.



Ricerca a profondità limitata

Per evitare che la ricerca in profondità si perda in un cammino infinito possiamo usare la **ricerca a profondità limitata**, in cui forniamo un limite di profondità **L**, e consideriamo tutti i nodi a profondità **L** come se non avessero successori.

La scelta del limite dovrebbe essere condizionata dalla *previsione* sulla profondità a cui si trova la soluzione. Se si sceglie male **L** l'algoritmo **non riuscirà a trovare la soluzione**, per cui è ancora **incompleto**.

La complessità temporale è quindi **$O(b^L)$** e quella spaziale è **$O(bL)$** .

La ricerca in profondità è una ricerca ad albero e quindi non possiamo evitare che in generale perda tempo su **cammini ridondanti**, ma possiamo eliminare i cammini ciclici al costo di un po' di tempo di calcolo in più: esaminando soltanto pochi collegamenti risalendo nella catena di parentela, riusciamo a catturare la maggior parte dei cicli.



Ricerca a profondità limitata: Algoritmo

```
function Ricerca-Profondità-Limitata(problema, L) returns un nodo soluzione o fallimento o soglia  
  frontiera  $\leftarrow$  una coda LIFO con Nodo(problema.StatoIniziale) come elemento iniziale  
  risultato  $\leftarrow$  fallimento  
  while not Vuota?(frontiera) do  
    nodo  $\leftarrow$  Pop(frontiera)  
    if nodo.Stato == problema.Obiettivo then return nodo  
    if Profondità(nodo) > L then  
      risultato  $\leftarrow$  soglia  
    else if not È-Ciclo(nodo) do  
      for each figlio in Espandi(problema, nodo) do  
        aggiungi figlio a frontiera  
  
  return risultato
```



Ricerca ad approfondimento iterativo

Per risolvere il problema di trovare il giusto valore di **L** si applica una semplice strategia: Provare ad effettuare una ricerca a profondità limitata per tutti i valori possibili di **L**: prima 0, poi 1, poi 2, e così via fino a quando si trova una soluzione, oppure la ricerca a profondità limitata restituisce il valore fallimento anziché il valore soglia.

```
function Ricerca-Approfondimento-Iterativo(problema) returns un nodo soluzione o fallimento
  for profondità = 0 to  $\infty$  do
    risultato  $\leftarrow$  Ricerca-Profondità-Limitata(problema, profondità)
    if risultato  $\neq$  soglia then return risultato
    #se risultato è diverso da soglia allora è la soluzione oppure fallimento
```



Ricerca ad approfondimento iterativo: osservazioni

L'**approfondimento iterativo** riunisce molti vantaggi offerti dalla ricerca in profondità e da quella in ampiezza.

Se **b** è il fattore di ramificazione e se la profondità della soluzione è **d**, allora il numero massimo di nodi espansi dall'algoritmo di ricerca iterativa in profondità è **b^d**

Complessità temporale: **$O(b^d)$** oppure **$O(b^m)$** se non esiste una soluzione (caso peggiore)

Come nella ricerca in profondità, i requisiti di memoria sono modesti: **$O(bd)$** quando esiste una soluzione a profondità **d**

Complessità temporale: **$O(bd)$** oppure **$O(bm)$** su spazi degli stati **finiti** senza soluzione.

Come la ricerca in ampiezza, la ricerca ad approfondimento iterativo è **ottima** per problemi in cui tutte le azioni hanno **lo stesso costo**, ed è **completa** su spazi degli **stati aciclici finiti**, o su qualsiasi spazio degli stati finito quando controlliamo i nodi per la presenza di cicli risalendo l'intero cammino.

